

Репозиторії аналітичних моделей

Transfer learning (індивідуальна частина) та reinforcement learning (групова частина)

Задача: класифікація речового забезпечення за фото

Дані

Fashion-MNIST (Zalando, відкритий): 10 категорій, зображення 28×28

Обмеження

Розмічених фото мало: від 10 до 100 на категорію

Питання

Чи дає готова модель, навчена на мільйонах фото, кращу точність, ніж навчання з нуля?

Протокол: test 2000 · validation 500 · пул 7500, з якого беремо малі вибірки; 3 повтори на кожен бюджет.

МОДЕЛЬ З РЕПОЗИТОРІЮ

MobileNetV2, навчена на ImageNet

1

Фіксоване джерело:
конкретний реліз ваг
Keras Applications

2

Перевірка SHA-256:
файл не пошкоджений
і не підмінений

3

Model card: дані, вхід,
ліцензія, обмеження

4

Узгодження входу:
28×28 сіре → 96×96
RGB, нормалізація [−1;
1]

2.26 млн параметрів · 1.28 млн навчальних зображень · 1000 класів ImageNet

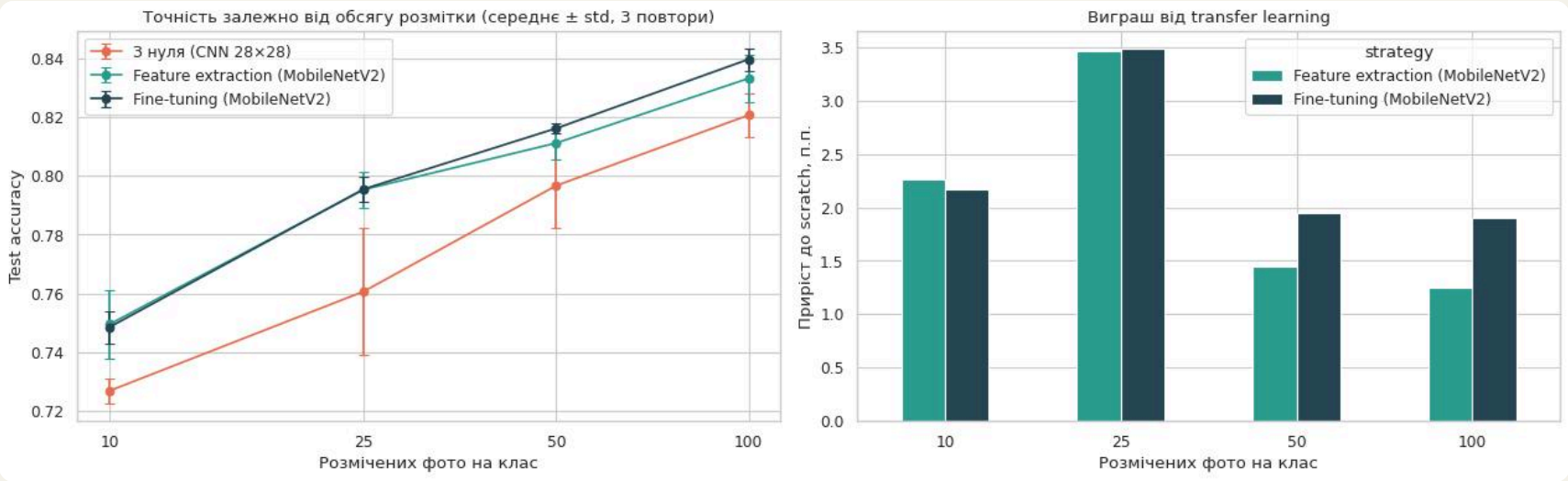
Три стратегії на однакових даних

Стратегія	Що навчається	Вартість
З нуля	Невелика CNN на 28×28 з випадкових ваг	9–25 s на запуск
Feature extraction	Лише нова голова на заморожених ознаках MobileNetV2	~7 s; ознаки один раз, ~30 s
Fine-tuning	Голова і верхні 30 шарів, lr 1e-5, BatchNorm заморожений	22–43 s на запуск

Бюджети розмітки: 10 · 25 · 50 · 100 фото на клас, по 3 повтори з різними вибірками.

Transfer learning кращий на всіх бюджетах

Фото/клас	З нуля	Feature ext.	Fine-tune
10	0.727	0.750	0.748
25	0.761	0.795	0.796
50	0.797	0.811	0.816
100	0.821	0.833	0.840



Середнє з 3 повторів. Приріст над навчанням з нуля: +1.2...+3.5 п.п.

+3.5

п.п. точності при 25 фото на клас
порівняно з навчанням з нуля

+36

п.п. recall для класу «свєтр»:
перенесення найбільше
допомагає класам із характерною
текстурою

7 s

навчання в режимі feature
extraction проти 22–43 s у fine-
tuning за майже тієї ж точності

Коли брати модель з репозиторію

- Мало розмітки: готова модель дає стабільний приріст і менший розкид результатів.
- Спершу feature extraction: дешево і майже так само точно; fine-tuning лише за більших бюджетів (+0.5–0.7 п.п.).
- Робота з репозиторієм — це дисципліна: фіксоване джерело, SHA-256, model card, узгоджений вхід.
- Зсув домену обмежує виграш: для фото техніки варто брати модель, ближчу до цільових даних.

Задача: маршрут колони у змінній обстановці

Карта 10×10

Від пункту постачання S до підрозділу G. Рубіж з двома переправами: короткою (17 кроків) і обхідною (27 кроків).

Змінні умови

Перед рейсом розвідка повідомляє, чи коротка переправа під вогнем (50 %). У русі 10 % шанс зносу колони.

Винагороди

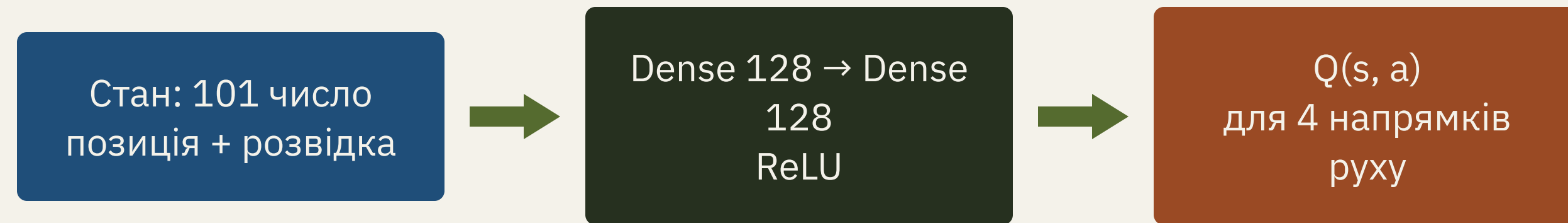
Крок –1 · крок у зоні вогню –25 · прибуття до G
+100

Мета агента

Їхати коротко, коли безпечно, і обходити, коли переправа під вогнем.

МЕТОД

Deep Q-Network на TensorFlow



Experience replay

Буфер 20 000 переходів, батчі по 64

Target network

Копія мережі оновлюється кожні 500 кроків

Навчання

1500 епізодів, ϵ 1.0 $\rightarrow$ 0.05, γ = 0.99, $\approx$ 90 s на CPU

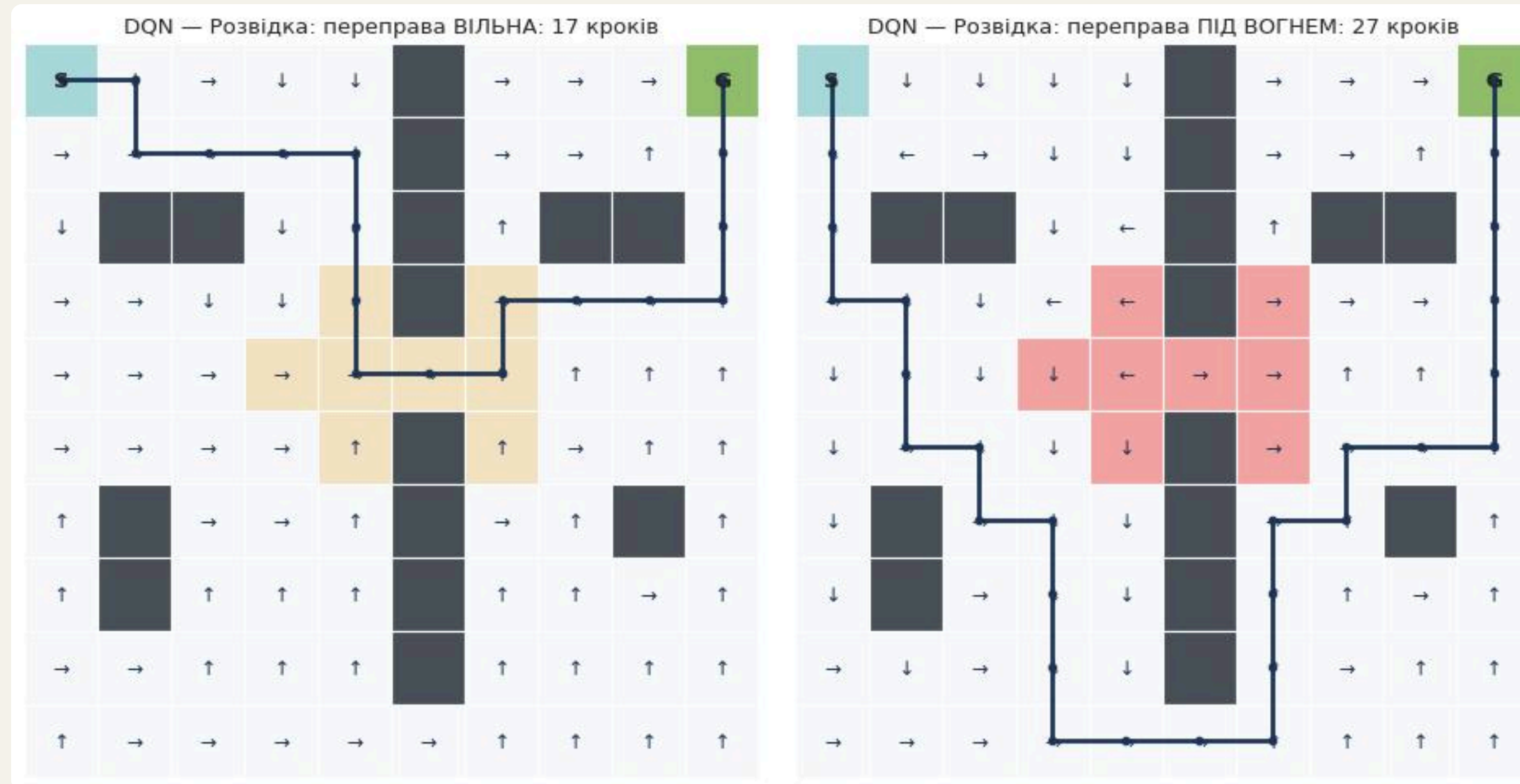
РЕЗУЛЬТАТИ · 1000 РЕЙСІВ, ОБИДВІ УМОВИ РОЗВІДКИ

DQN кращий за кожне фіксоване правило

Агент	Винагорода	Кроків	Під вогнем
Правило: завжди коротка переправа	10.7	19.0	2.82
Правило: завжди обхід	67.5	30.1	0.10
Табличний Q-learning (еталон)	75.1	24.6	0.01
DQN-агент	74.6	24.6	0.03

Успіх (прибуття до G): 100 % у всіх агентів, крім випадкової політики (8 %). «Під вогнем» — середня кількість кроків у зоні обстрілу за рейс.

Одна мережа — два рішення залежно від розвідки



Переправа вільна: коротким шляхом, 17 кроків.

Під вогнем: обхід через безпечну переправу, 27 кроків.

Стрілки — дія в кожній клітині: якщо колону знесе, агент знає, як повернутися.

Агент оптимізує те, що йому задали

- DQN поєднує швидкість і обережність: винагорода 74.6 проти 67.5 («завжди обхід») і 10.7 («завжди коротко»).
- Ціну ризику задає командир: при штрафі 0...–5 агент іде через зону вогню, з –25 — стабільно обходить.
- DQN масштабується на складніші стани (час, пальне, кілька загроз), де таблиця вже не працює.
- Далі: кілька динамічних загроз, хибна розвідка, порівняння з Actor-Critic (PPO).